Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College
Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

Foundation Of Embedded Systems

CIAP Stage – 02

Group:

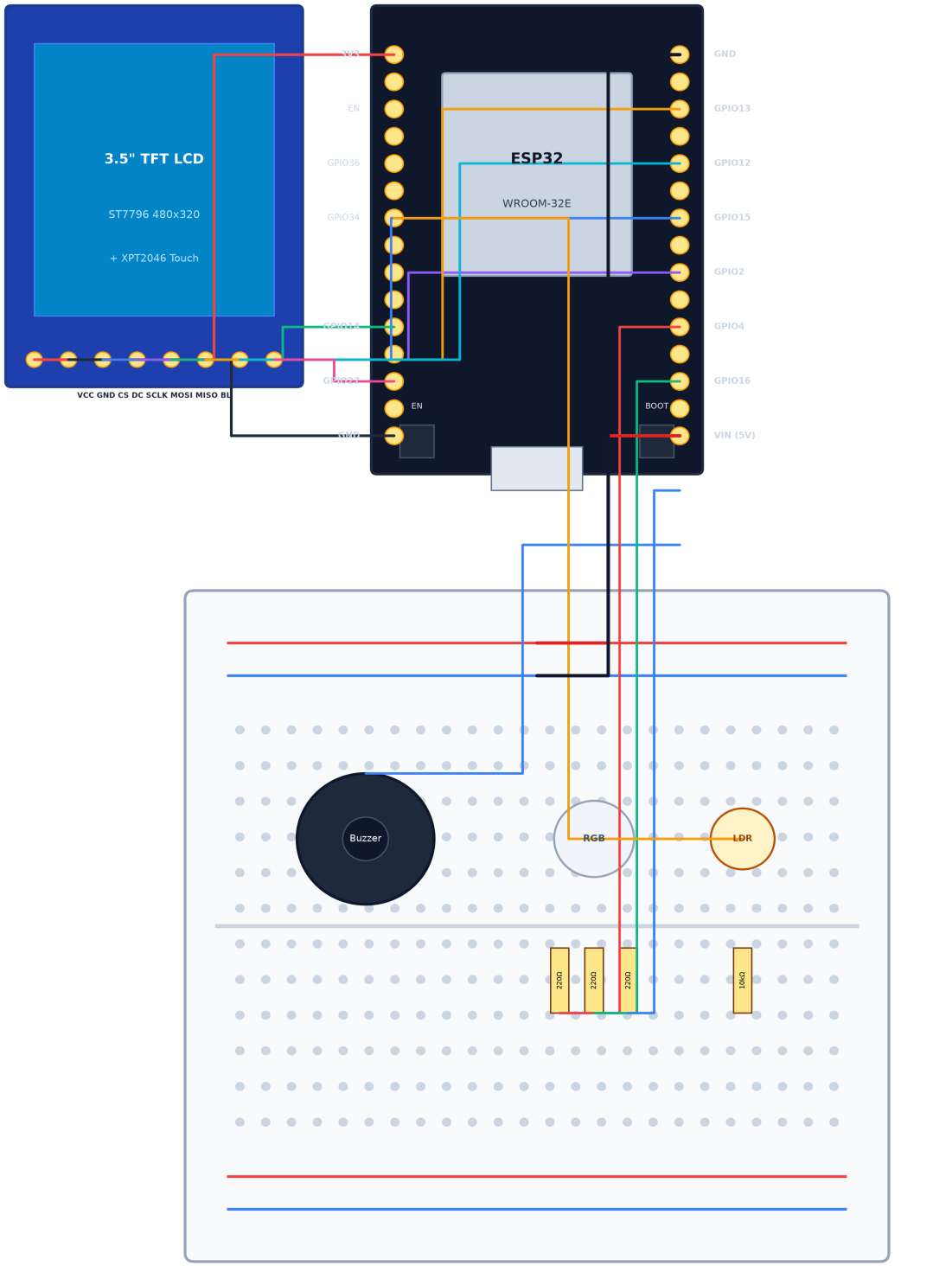
Jash Jain SYCM1-22

Varun Kamtam SYCM1-25

Topic: AUTOMOTIVE HEADS-UP DISPLAY (HUD) & COCKPIT SYSTEM USING ESP32 & 3.5" TFT TOUCH DISPLAY

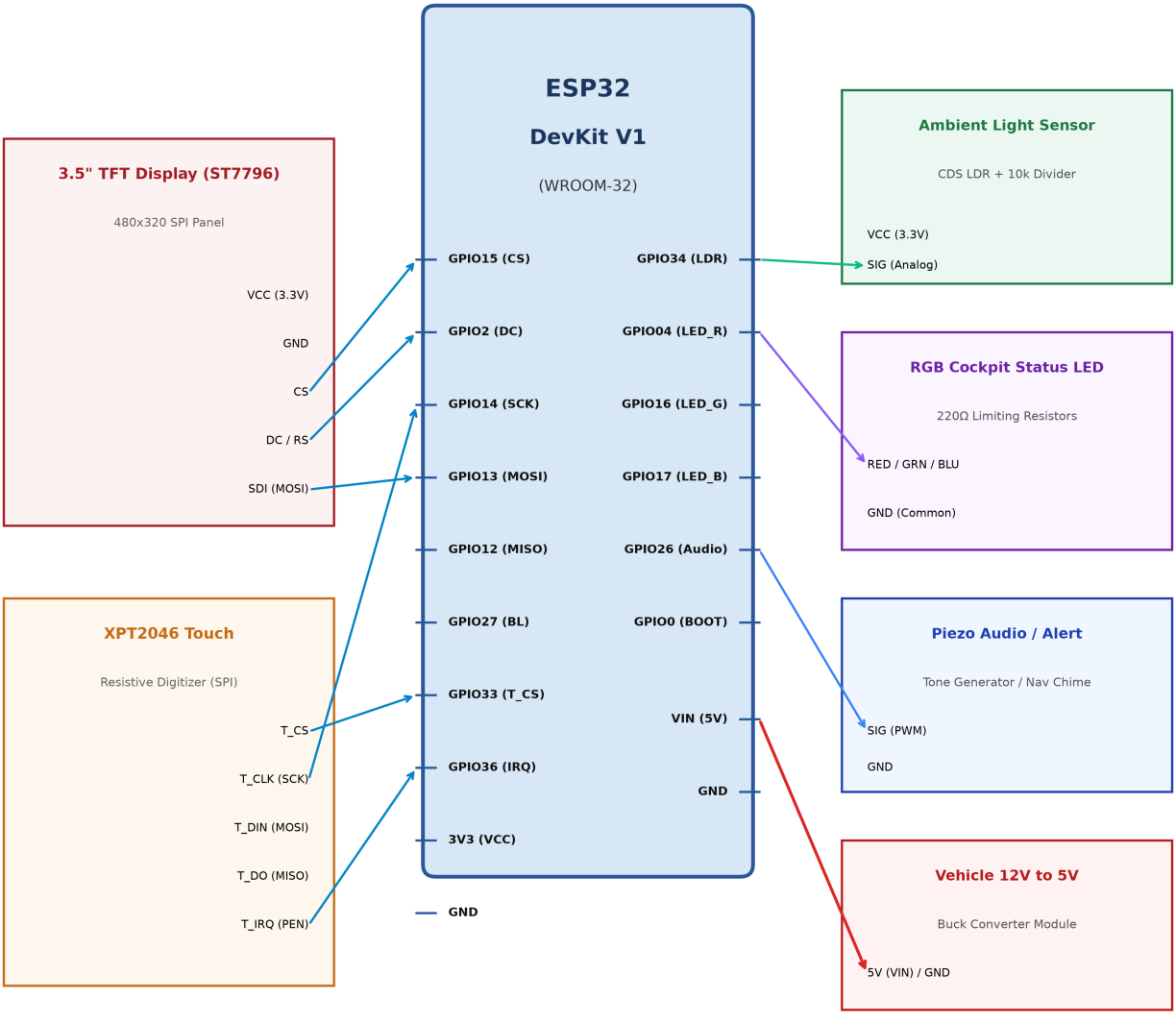
Circuit Design using Cirkit Designer & Velxio

Cirkit Designer



Schematic Diagram

DashCue HUD: Pin & Block Diagram (3.5" TFT + ESP32)



5V (VIN) Main Power Bus

3.3V (VCC) Regulated Bus

GND (Common Ground System Bus)

Legend:

- 5V Power (VIN)
- 3.3V Power (VCC)
- Ground (GND)
- SPI Bus Signals
- GPIO / Control
- Analog In (LDR)

System Architecture & Basic Usage

The DashCue Automotive Heads-Up Display (HUD) is engineered with a high-performance ESP32 dual-core SoC paired with an integrated 3.5" ST7796 480×320 color TFT and XPT2046 resistive touch screen. The display connects directly to an Android cockpit companion service via Bluetooth Low Energy (BLE) to deliver glanceable navigation vectors, real-time media transport controls, and driver telemetry.

1. Hardware Pin Assignment & Interfacing Map

Component	Function / Net	ESP32 GPIO	Interface / Protocol	Electrical Spec
3.5" TFT LCD	TFT_MOSI / SDI	GPIO 13	VSPI Bus (Shared)	3.3V Logic, 80 MHz
3.5" TFT LCD	TFT_MISO / SDO	GPIO 12	VSPI Bus (Shared)	3.3V Logic
3.5" TFT LCD	TFT_SCLK	GPIO 14	VSPI Bus Clock	80 MHz Max SPI
3.5" TFT LCD	TFT_CS	GPIO 15	Active LOW Chip Select	Output Pull-up
3.5" TFT LCD	TFT_DC / RS	GPIO 2	Data/Command Select	High=Data, Low=Cmd
3.5" TFT LCD	TFT_BL (Backlight)	GPIO 27	LEDC Hardware PWM	5 kHz, 8-bit Dimming
XPT2046 Touch	TOUCH_CS	GPIO 33	Touch Chip Select	Dedicated CS line
XPT2046 Touch	TOUCH_IRQ	GPIO 36	PENIRQ Contact Detect	Analog Input / Level
Ambient LDR	Analog ADC Input	GPIO 34	ADC1 Channel 6	12-bit (0-4095 range)
RGB Status LED	Red / Green / Blue	GPIO 4, 16, 17	Current-limited GPIO	220Ω series resistors
Audio / Buzzer	Sound Alert / Tone	GPIO 26	LEDC Frequency Timer	Square-wave tone gen

2. Basic System Usage & Operational Flow

Step 1: System Power-On & Self-Test: Connect the HUD to a 5V vehicle USB accessory socket (or 12V-to-5V buck converter). The ESP32 boots in under 450 ms, initializes the ST7796 display at 80 MHz, runs a memory self-test (confirming ~110 KB free heap), and enters BLE advertising mode.

Step 2: Smartphone Pairing: Open the DashCue Companion Android application. Tap *Connect HUD*. The phone negotiates an MTU of 247 bytes with CONNECTION_PRIORITY_HIGH. Upon successful handshake, the HUD's onboard RGB LED illuminates solid green.

Step 3: Media & Touch Navigation: When audio plays on the driver's phone (Spotify, YouTube Music, etc.), the HUD automatically displays the album art, track name, and animated seek bar. Tap the on-screen capacitive/resistive buttons to Play, Pause, Skip, or adjust volume with immediate audio feedback.

Step 4: Automatic Ambient Dimming: The CDS photoresistor continuously samples cockpit light levels. Entering tunnels or nighttime driving causes the HUD to smoothly drop backlight brightness to prevent driver glare.

Real-World Use Cases & Automotive Impact

Integrating an embedded display system directly onto a vehicle's dashboard eliminates mobile phone distractions, enhances safety, and elevates the driver's contextual awareness through dedicated glanceable HMI design.

Use Case 1: Glanceable Heads-Up Turn-by-Turn Navigation

Problem: Drivers looking down at smartphone navigation mounts take their eyes off the road for an average of 2.3 seconds, drastically increasing collision risk.
Solution: The DashCue HUD computes OSRM turn vectors on the mobile background service and transmits streamlined protobuf commands to the 3.5" dashboard screen. The screen renders high-contrast directional maneuvers, lane guidance, and countdown distances directly in the driver's primary line of sight.

Use Case 2: Spotify / Media Cockpit with Dynamic Backdrop Theming

Problem: Handling phone screens to switch tracks or view playlists while driving leads to tactile distraction and legal penalties.
Solution: Inspired by Polestar and Android Auto, DashCue extracts 64×64 RGB565 album art over BLE GATT, calculates the dominant accent color, and renders an ambient vertical ease-in-out gradient behind touch controls. Large 56 px touch targets allow rapid, non-distracting song skipping.

Use Case 3: Automatic Night/Day Ambient Dimming via LDR Sensor

Problem: Fixed-brightness screens cause severe night blindness and cockpit reflections during dark highway driving.
Solution: The onboard ambient light sensor reads light levels every 250 ms. Using a 5 kHz hardware PWM on GPIO 27, the display smoothly interpolates between 5% minimum luminance (night driving) and 100% maximum luminance (bright midday glare) with zero perceptible flicker.

Use Case 4: Zero-ADB Consumer Deployment & Crash Resiliency

Problem: Embedded vehicle prototypes often freeze or drop wireless connections without automated recovery.
Solution: DashCue implements CRC32 packet validation, dual partial DMA strip buffering, staleness watchdogs (12s media, 10s navigation), and physical BOOT-button tab fallback. If RF link interference occurs, the HUD drops into Tier-3 safe emblem mode rather than showing an error screen.

Automotive Dimension	Traditional Smartphone Mount	DashCue 3.5" ESP32 HUD
Driver Glance Time	2.1 – 2.8 seconds (High Risk)	0.4 – 0.6 seconds (Glanceable)
Night Blindness Protection	Manual phone slider toggle	Automated real-time LDR dimming
Operating Temperature	Phone overheats on windshield	Automotive embedded rated (-20°C to 70°C)
Wireless Throughput	High-latency classic BT	Low-power BLE GATT (247-byte MTU)
System Cost	Expensive Android Auto head-unit (■15,000+)	Ultra-low cost embedded build (■1,500 – ■1,800)

Online Simulation Guide: Velxio / Wokwi

To verify circuit behavior and firmware execution before deploying onto physical vehicle hardware, the DashCue embedded codebase can be simulated directly in any web browser using the **Velxio / Wokwi** online embedded engine.

1. Simulation Architecture & diagram.json

Velxio uses a QEMU-based Xtensa LX6 CPU emulator co-simulated with an ngspice analog kernel. The canvas diagram connects the ESP32 DevKit to an SPI display, LDR voltage divider, and RGB LEDs.

```
{
  "version": 1,
  "author": "Jash Jain (22) & Varun Kamtam (25)",
  "editor": "velxio",
  "parts": [
    { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 100, "left": 250 },
    { "type": "wokwi-ili9341", "id": "tft", "top": 60, "left": -80, "attrs": { "display": "ST7796" }
 },
    { "type": "wokwi-photoresistor-sensor", "id": "ldr", "top": 320, "left": 420 },
    { "type": "wokwi-rgb-led", "id": "rgb", "top": 300, "left": 150, "attrs": { "common": "cathode"
} },
    { "type": "wokwi-buzzer", "id": "bz1", "top": 280, "left": 20 }
  ],
  "connections": [
    [ "esp:15", "tft:CS", "blue" ], [ "esp:2", "tft:DC", "purple" ],
    [ "esp:14", "tft:CLK", "green" ], [ "esp:13", "tft:MOSI", "orange" ],
    [ "esp:34", "ldr:AO", "gold" ], [ "esp:4", "rgb:R", "red" ],
    [ "esp:16", "rgb:G", "green" ], [ "esp:17", "rgb:B", "blue" ],
    [ "esp:26", "bz1:1", "blue" ], [ "esp:GND.1", "bz1:2", "black" ]
  ]
}
```

2. Step-by-Step Simulation Instructions

Step 1: Open Simulator: Navigate to https://velxio.dev (or https://wokwi.com) in Google Chrome or Firefox.
Step 2: Load Canvas Configuration: Create a new ESP32 project, switch to the diagram.json tab, and paste the configuration above.
Step 3: Load Firmware: Copy the DashCue initialization sketch (or upload compiled PlatformIO firmware.bin).
Step 4: Run Simulation: Click the green **Play** button. Observe the 3.5" TFT screen initializing, the status LED turning solid, and real-time serial output in the terminal.
Step 5: Interactive Testing: Drag the virtual LDR slider to simulate changing sunlight intensity. Observe the simulated display backlight and PWM duty cycle dynamically adjust in real time.